

Problem1

April 19, 2026

1 AA 203 — Homework 1

1.1 Problem 1.1 — Gradient descent and line search

1.1.1 a)

Using the NOC Theorem, given that f is twice differentiable, $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive semi-definite for any candidate local minimum x^* . Since $f(x) \in C^2$,

$$\nabla f(x^*) = Qx^* - b = 0 \quad \text{and} \quad \nabla^2 f(x^*) = Q \rightarrow (\text{positive-definite matrix})$$

$$\Rightarrow \boxed{x^* = Q^{-1}b}$$

Let's use Q is symmetric positive-definite $\Rightarrow$ this also implies x^* is actually a strict local minimum.

Prove x^* is a global minimum - f is defined on $\mathbb{R}^n$ that is open - $\forall x \in \mathbb{R}^n$, $\nabla^2 f(x) = Q$ is positive-definite $\Rightarrow f$ is strictly convex - Therefore the NOC is actually a necessary and sufficient condition, proving $x^* = Q^{-1}b$ is a unique global minimum.

1.1.2 b) Using Newton method

$$\begin{aligned} x^{(k+1)} &= x^{(k)} - [\nabla^2 f(x^{(k)})]^{-1} \nabla f(x^{(k)}) \\ &= x^{(k)} - Q^{-1}(Qx^{(k)} - b) \\ &= x^{(k)} - x^{(k)} + Q^{-1}b \\ &= Q^{-1}b = x^* \end{aligned}$$

$\Rightarrow$ we get to x^* in one step.

The downside is that we need to compute Q^{-1} which on average is $\mathcal{O}(n^3)$ in time and $\mathcal{O}(n^2)$ in memory. For $n \gg 10^4$, which can be very computationally challenging. Also this inversion of Q is unstable, which is only possible due to the assumed structure of the matrix (symmetric definite positive).

1.1.3 c)

$$\|Sx\|_2 = \|U\Sigma U^\top x\|_2 = \|\Sigma U^\top x\|_2 \quad (\text{because } U \text{ is orthogonal}), \quad \forall x \in \mathbb{R}^n.$$

$$\forall z \in \mathbb{R}^n. \Sigma z = \begin{pmatrix} \mu_1 & & \\ & \ddots & \\ & & \mu_n \end{pmatrix} \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} = \begin{pmatrix} \mu_1 z_1 \\ \vdots \\ \mu_n z_n \end{pmatrix}$$

Since for all i , $\mu_i^2 \leq (\max_{j \in \{1, \dots, n\}} |\mu_j|)^2$, we get

$$\|\Sigma z\|_2 = \sqrt{\sum_i \mu_i^2 z_i^2} \leq \max_{i \in \{1, \dots, n\}} |\mu_i| \sqrt{\sum_i z_i^2} = \max_{i \in \{1, \dots, n\}} |\mu_i| \|z\|_2$$

$$\Rightarrow \|\Sigma z\|_2 \leq \max_{i \in \{1, \dots, n\}} |\mu_i| \|z\|_2$$

$\forall x \in \mathbb{R}^n$,

$$\|Sx\|_2 = \|\Sigma U^\top x\|_2 \leq \max_{i \in \{1, \dots, n\}} |\mu_i| \|U^\top x\|_2 = \max_{i \in \{1, \dots, n\}} |\mu_i| \|x\|_2 \quad \text{because } U^\top \text{ is orthogonal}$$

$$\text{Thus } \boxed{\|Sx\|_2 \leq \max_{i \in \{1, \dots, n\}} |\mu_i| \|x\|_2}$$

1.1.4 d)

Q is a symmetric, definite positive matrix $\rightarrow$ spectral theorem implies

$$\exists U \in \mathbb{R}^{n \times n} \text{ orthogonal s.t. } Q = U \Sigma_Q U^\top \quad \text{where } \Sigma_Q = \text{diag}(\lambda_1, \dots, \lambda_n).$$

Note $e_i = (0, \dots, \underset{\substack{\uparrow \\ \text{ith pos}}}{1}, \dots, 0)^\top$ the canonical base of $\mathbb{R}^n$, and $v_i = U e_i$.

$$\begin{aligned} (I - \eta Q) v_i &= (I - \eta Q) U e_i \\ &= U e_i - \eta U \Sigma_Q U^\top U e_i \\ &= U e_i - \eta U \Sigma_Q e_i \\ &= U e_i - \eta \lambda_i U e_i \\ &= (1 - \eta \lambda_i) U e_i \\ &= (1 - \eta \lambda_i) v_i \quad (**) \end{aligned}$$

Thus $(I - \eta Q) v_i = (1 - \eta \lambda_i) v_i$, and (v_i) , $i = 1, \dots, n$ is another base of $\mathbb{R}^n$ since U is orthogonal.
(**)

For each i , $(1 - \eta \lambda_i)$ is an eigenvalue of $I - \eta Q$ due to (**).

1.1.5 e)

For $k \geq 0$,

$$\begin{aligned}
\delta^{(k+1)} &= \|x^{(k+1)} - x^*\|_2 \\
&= \|x^{(k)} - \eta(Qx^{(k)} - b) - \underbrace{[x^* - \eta(Qx^* - b)]}_{=0}\|_2 \quad (\text{since } \nabla f(x^*) = Qx^* - b = 0) \\
&= \|[I - \eta Q]x^{(k)} + \eta b - [I - \eta Q]x^* - \eta b\|_2 \\
&= \|[I - \eta Q][x^{(k)} - x^*]\|_2 \\
&\leq \max_{i \in \{1, \dots, n\}} |1 - \eta \lambda_i| \cdot \|x^{(k)} - x^*\|_2
\end{aligned}$$

using question c) (Note that since Q is symmetric “ $S = I - \eta Q$ ” is also symmetric, so c) applies in here, and in d) we computed the eigen values).

$$\implies \delta^{(k+1)} \leq \gamma(\eta) \delta^{(k)}$$

By recursion, since all these are positive we have:

$$\forall k \geq 0, \quad \delta^{(k)} \leq \gamma(\eta)^k \cdot \underbrace{\delta^{(0)}}_{= \|x^{(0)} - x^*\|_2 = \delta_0}$$

1.1.6 f)

$d^{(k)} = -\nabla f(x^{(k)})$. Define $g(\eta) = f(x^{(k)} + \eta d^{(k)})$.

$$\begin{aligned}
g'(\eta) &= (x^{(k)})^\top Q d^{(k)} + \eta d^{(k)\top} Q d^{(k)} - b^\top d^{(k)} \\
&= d^{(k)\top} (Qx^{(k)} - b) + \eta d^{(k)\top} Q d^{(k)}
\end{aligned}$$

Let's find η^* : η unconstrained optimization problem, so

$$\min_{\eta \geq 0} g(\eta)$$

since we have a 1D convex problem defined on the positive reals, a necessary and sufficient condition is

$$g'(\eta) = 0$$

$$0 = d^{(k)\top} (Qx^{(k)} + \eta d^{(k)} - b) = d^{(k)\top} [(Qx^{(k)} - b) + \eta Qd^{(k)}]$$

$$0 = d^{(k)\top} (-d^{(k)}) + \eta d^{(k)\top} Q d^{(k)} \quad \left(= -\underbrace{\|d^{(k)}\|_2^2}_2 \right)$$

$$\eta^* = \frac{\|d^{(k)}\|_2^2}{d^{(k)\top} Q d^{(k)}}$$

η^* happens to be the global minimum of g defined on $\mathbb{R}$, hence it is also a global minimum of g on $\mathbb{R}^+$.

$$\Rightarrow \eta^{(k)} = \arg \min_{\eta \geq 0} f(x^{(k)} + \eta d^{(k)}) = \frac{\|d^{(k)}\|_2^2}{d^{(k)\top} Q d^{(k)}}$$

1.1.7 g)

$$f(x) = \frac{1}{2} x_1^2 + \gamma x_2^2 \cdot \frac{1}{2} = \frac{1}{2} x^\top Q x - b^\top x$$

For

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & \gamma \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Please see code.

```
[1]: import numpy as np
import matplotlib.pyplot as plt

def gd_const(Q, x0, eta, n_iters=60):
    xs = [np.asarray(x0, dtype=float)]
    for _ in range(n_iters):
        xs.append(xs[-1] - eta * (Q @ xs[-1]))
    return np.array(xs)

def gd_linesearch(Q, x0, n_iters=60, tol=1e-12):
    xs = [np.asarray(x0, dtype=float)]
    for _ in range(n_iters):
        d = -Q @ xs[-1]
        if np.linalg.norm(d) < tol:
            break
        eta = (d @ d) / (d @ Q @ d)
        xs.append(xs[-1] + eta * d)
    return np.array(xs)

def plot_traj(ax, Q, traj, title, lim=6):
    g = np.linspace(-lim, lim, 300)
    X, Y = np.meshgrid(g, g)
    Z = 0.5 * (Q[0, 0] * X**2 + Q[1, 1] * Y**2)
    ax.contour(X, Y, Z, levels=25, cmap="viridis", alpha=0.6)
    ax.plot(traj[:, 0], traj[:, 1], "r.-", lw=1, ms=4)
    ax.plot(0, 0, "k*", ms=12)
    ax.set_title(title)
    ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
    ax.set_aspect("equal")
    ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim)

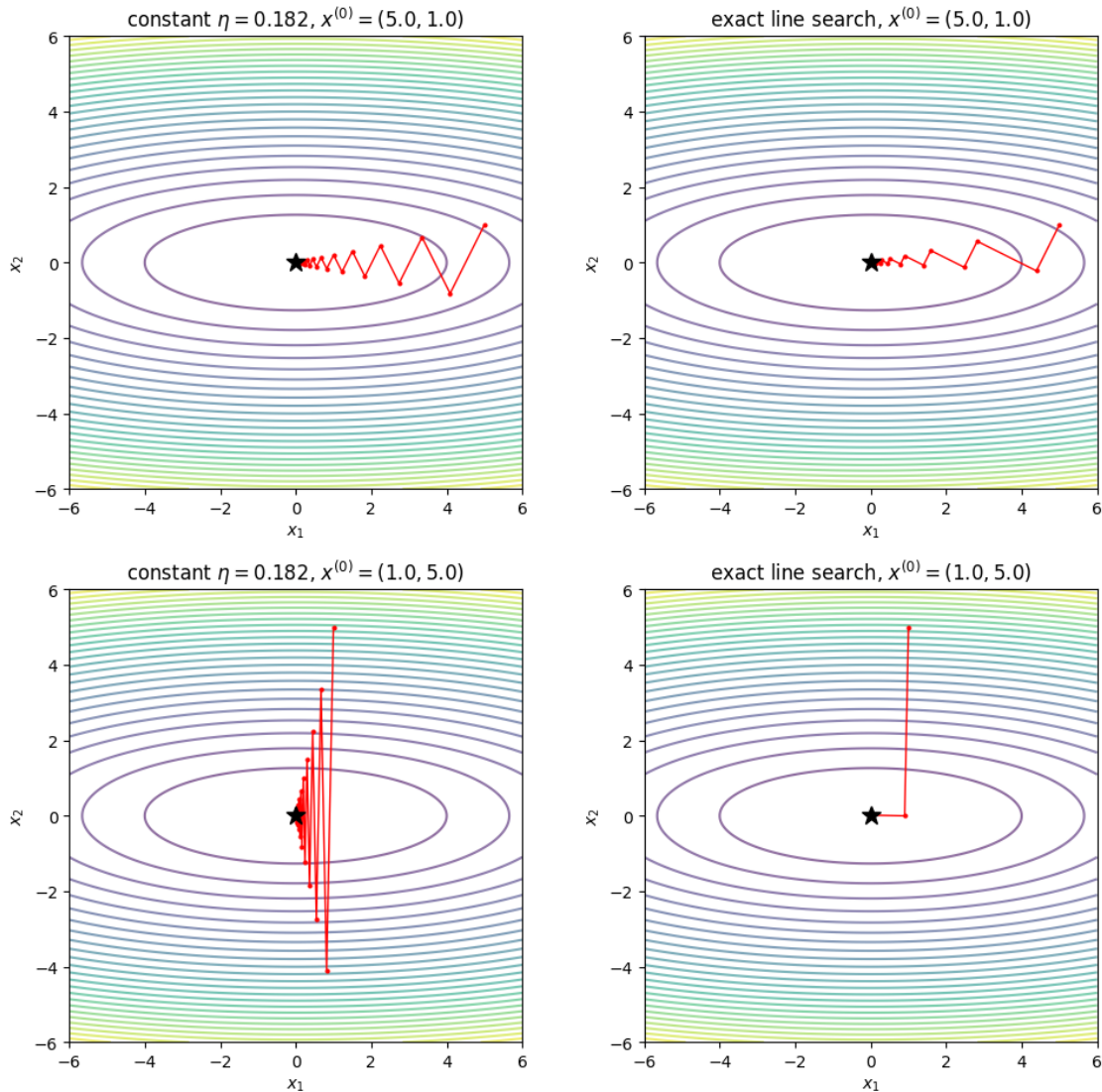
starts = [(5.0, 1.0), (1.0, 5.0)]
```

```

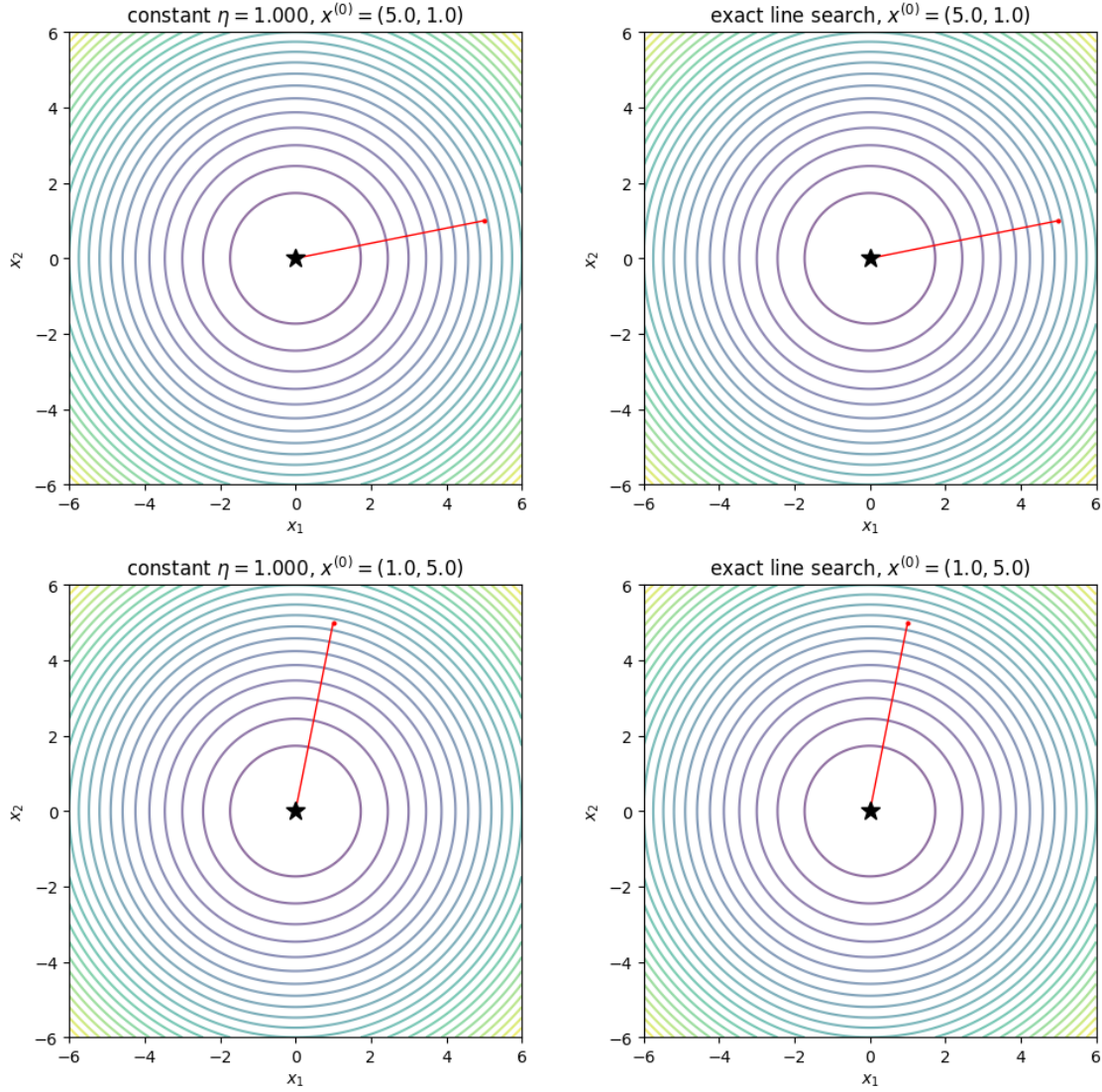
for gamma in (10, 1):
    Q = np.diag([1.0, float(gamma)])
    eta = 2.0 / (1.0 + gamma) # 2 / (lambda_min + lambda_max)
    fig, axes = plt.subplots(2, 2, figsize=(10, 10))
    for i, x0 in enumerate(starts):
        tc = gd_const(Q, x0, eta=eta)
        tl = gd_linesearch(Q, x0)
        plot_traj(axes[i, 0], Q, tc, f"constant  $\eta={eta:.3f}$ ,  $x^{(0)}={x0}$ ")
        plot_traj(axes[i, 1], Q, tl, f"exact line search,  $x^{(0)}={x0}$ ")
    fig.suptitle(fr"$\gamma = {gamma}$", fontsize=14)
    plt.tight_layout()
    plt.show()

```

$\gamma = 10$



$\gamma = 1$



Observations.

Exact line search always reaches $x^* = (0, 0)$. For $\gamma = 10$ the iterates zig-zag: consecutive line-search directions are orthogonal (since $\nabla f(x^{(k+1)})^\top d^{(k)} = g'(\eta^{(k)}) = 0$), and the bigger the condition number γ , the more severe the zig-zag. For $\gamma = 1$ the level sets are circles, the gradient points straight at x^* , and we converge in a single iteration from any starting point.

Constant step size needs $\eta < 2/\lambda_{\max} = 2/\gamma$ to be stable (from (e), $\gamma(\eta) < 1$). The best constant step is $\eta^* = 2/(\lambda_{\min} + \lambda_{\max}) = 2/(1 + \gamma)$, with contraction factor $(\gamma - 1)/(\gamma + 1)$. For $\gamma = 10$ that is $9/11 \approx 0.82$, so progress is slow and we see the same zig-zag as with exact line search. If η is too large the iteration diverges; too small and progress crawls. For $\gamma = 1$, $\eta^* = 1$ also gives one-step

convergence.

Problem 1.2

April 19, 2026

0.1 Problem 1.2 — LQR as a QP

0.1.1 a)

From $x_{t+1} = Ax_t + Bu_t$, expanding recursively:

$$x_1 = Ax_0 + Bu_0$$

$$x_2 = A(Ax_0 + Bu_0) + Bu_1 = A^2x_0 + ABu_0 + Bu_1$$

By recursion we can write

$$x_t = A^t x_0 + \sum_{i=0}^{t-1} A^{t-1-i} B u_i$$

Writing $\mathbf{x} = (x_0, x_1, \dots, x_T)^\top$ and $u = (u_0, u_1, \dots, u_{T-1})^\top$, we have

$$\mathbf{x} = Mx_0 + Gu$$

where

$$M = \begin{pmatrix} I \\ A \\ A^2 \\ \vdots \\ A^T \end{pmatrix}, \quad G = \begin{pmatrix} 0 & 0 & \dots & 0 \\ B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & & \ddots & \vdots \\ A^{T-1}B & A^{T-2}B & \dots & B \end{pmatrix}.$$

We note $H = \text{blkdiag}(Q, Q, \dots, Q, Q_T)$ and $L = \text{blkdiag}(R, \dots, R)$, so that

$$J(u) = \mathbf{x}^\top H \mathbf{x} + u^\top L u.$$

Substituting $\mathbf{x} = Mx_0 + Gu$:

$$\begin{aligned} J(u) &= (Mx_0 + Gu)^\top H (Mx_0 + Gu) + u^\top L u \\ &= x_0^\top M^\top H M x_0 + x_0^\top M^\top H G u + u^\top G^\top H M x_0 + u^\top G^\top H G u + u^\top L u \end{aligned}$$

The two cross terms are scalars, so $x_0^\top M^\top H G u = (x_0^\top M^\top H G u)^\top = u^\top G^\top H^\top M x_0$, and they combine into $2(G^\top H^\top M x_0)^\top u$:

$$J(u) = x_0^\top M^\top H M x_0 + 2(G^\top H^\top M x_0)^\top u + u^\top (G^\top H G + L) u.$$

This is exactly $\frac{1}{2} u^\top \bar{Q} u - \bar{b}^\top u + \text{const}$, so matching:

$$\boxed{\bar{Q} = 2(G^\top H G + L)}, \quad \boxed{\bar{b} = -2 G^\top H^\top M x_0}.$$

$\bar{Q}$ is positive-definite since $L \succ 0 \implies G^\top H G + L \succ 0$, so the QP has a unique minimiser $u^* = \bar{Q}^{-1} \bar{b}$.

0.1.2 b)

The QP from (a) is $\min_u \frac{1}{2} u^\top \bar{Q} u - \bar{b}^\top u$ with $\bar{Q} \succ 0$, so exactly the same setup as Problem 1.1. By 1.1(b), Newton's method converges in one step:

$$u^* = \bar{Q}^{-1} \bar{b}.$$

The optimal cost is then

$$J(u^*) = \frac{1}{2} u^{*\top} \bar{Q} u^* - \bar{b}^\top u^* + x_0^\top M^\top H M x_0.$$

Please see code.

```
[1]: import numpy as np
from scipy.linalg import block_diag

# Given data
n, m, T = 2, 1, 20
A = np.array([[1, 1], [0, 1]], dtype=float)
B = np.array([[0], [1]], dtype=float)
x0 = np.array([[1], [0]], dtype=float)
QT = 10 * np.eye(n)
Q = np.eye(n)
R = np.eye(m)

# Build M and G
M = np.vstack([np.linalg.matrix_power(A, t) for t in range(T + 1)])

G = np.zeros((n * (T + 1), m * T))
for t in range(1, T + 1):
    for i in range(t):
        G[t*n:(t+1)*n, i*m:(i+1)*m] = np.linalg.matrix_power(A, t - 1 - i) @ B

# Build H and L
H = block_diag(*([Q] * T + [QT]))
```

```

L = block_diag(*([R] * T))

# QP matrices
Qbar = 2 * (G.T @ H @ G + L)
bbar = -2 * G.T @ H.T @ M @ x0

# Newton: one step
u_star = np.linalg.solve(Qbar, bbar)

# Optimal cost:  $J(u^*) = 1/2 u^{*T} Qbar u^* - bbar^T u^* + x0^T M^T H M x0$ 
const = (x0.T @ M.T @ H @ M @ x0).item()
J_star = (0.5 * u_star.T @ Qbar @ u_star - bbar.T @ u_star).item() + const

print(f"J(u*) = {J_star:.6f}")

```

J(u*) = 2.947123

Problem1.3

April 19, 2026

0.1 Problem 1.3 — Extremal curves

0.1.1 1.3

Using the Euler-Lagrange equation in the Fundamental Theorem of Calculus of Variations as a necessary condition for an optimal curve:

$$\frac{\partial}{\partial x} \left(\frac{1}{2} \dot{x}(t)^2 + 5x(t)\dot{x}(t) + x(t)^2 + 5x(t) \right) - \frac{d}{dt} \frac{\partial}{\partial \dot{x}} \left(\frac{1}{2} \dot{x}(t)^2 + 5x(t)\dot{x}(t) + x(t)^2 + 5x(t) \right) = 0$$

$$5\dot{x}(t) + 2x(t) + 5 - \frac{d}{dt} [\dot{x}(t) + 5x(t)] = 0$$

$$5\dot{x}(t) + 2x(t) + 5 - \ddot{x}(t) - 5\dot{x}(t) = 0$$

$$\boxed{\ddot{x}(t) - 2x(t) = 5}$$

General solution for $A, B \in \mathbb{R}$:

$$x^*(t) = A \cosh(\sqrt{2}t) + B \sinh(\sqrt{2}t) - \frac{5}{2}$$

with BC $x^*(0) = 1$ and $x^*(1) = 3$.

$$x^*(0) = A - \frac{5}{2} = 1 \quad \Rightarrow \quad A = \frac{7}{2}$$

$$x^*(1) = \frac{7}{2} \cosh(\sqrt{2}) + B \sinh(\sqrt{2}) - \frac{5}{2} = 3 \quad \Rightarrow \quad B = \frac{\frac{11}{2} - \frac{7}{2} \cosh(\sqrt{2})}{\sinh(\sqrt{2})}$$

$$\boxed{x^*(t) = \frac{7}{2} \cosh(\sqrt{2}t) + \frac{11 - 7 \cosh(\sqrt{2})}{2 \sinh(\sqrt{2})} \sinh(\sqrt{2}t) - \frac{5}{2}}$$

Please see plot.

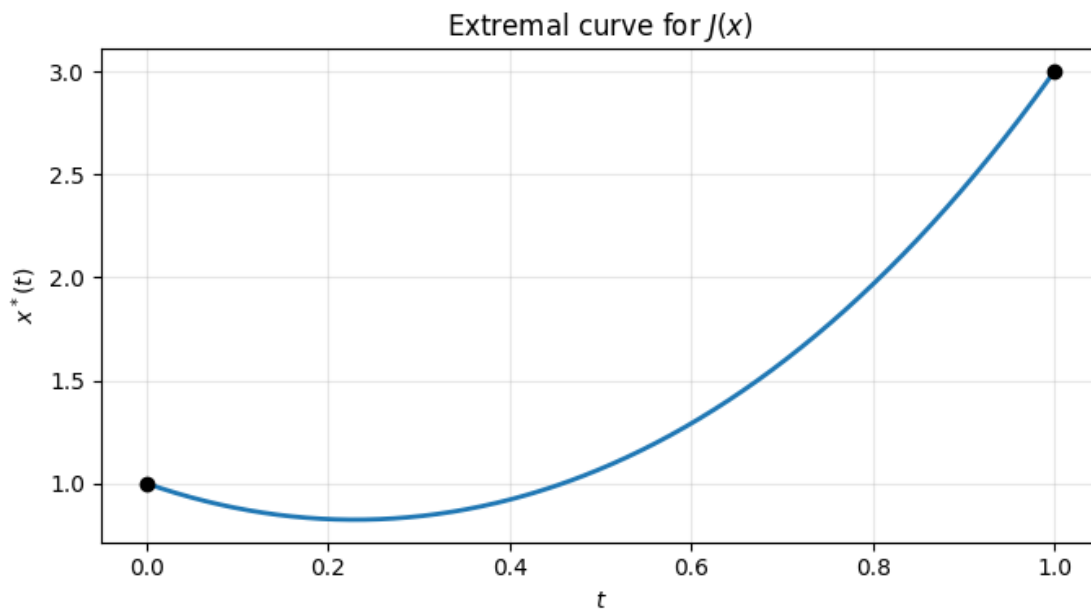
```
[1]: import numpy as np
import matplotlib.pyplot as plt

A = 7 / 2
B = (11/2 - (7/2) * np.cosh(np.sqrt(2))) / np.sinh(np.sqrt(2))

def x_star(t):
    return A * np.cosh(np.sqrt(2) * t) + B * np.sinh(np.sqrt(2) * t) - 5/2

t = np.linspace(0, 1, 500)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t, x_star(t), lw=2)
ax.scatter([0, 1], [1, 3], color='k', zorder=5)
ax.set_xlabel(r'$t$')
ax.set_ylabel(r'$x^*(t)$')
ax.set_title(r'Extremal curve for $J(x)$')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Problem 1.4

April 19, 2026

1 AA 203 — Homework 1

1.1 Problem 1.4 — Zermelo's ship

1.1.1 a)

Minimum-time objective: $J = \int_{t_0}^{t_1} 1 \, dt = t_1 - t_0$, so the Hamiltonian is

$$H = 1 + p_x \left(v \cos \theta(t) + \frac{v}{h} y(t) \right) + p_y v \sin \theta(t)$$

State equations (dynamic constraints):

$$\dot{x}^*(t) = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} v \cos \theta^* + \frac{v}{h} y \\ v \sin \theta^* \end{bmatrix}$$

Costate (adjoint) equations:

$$\dot{p}^*(t) = \begin{bmatrix} -\frac{\partial H}{\partial x} \\ -\frac{\partial H}{\partial y} \end{bmatrix} = \begin{bmatrix} 0 \\ -p_x \frac{v}{h} \end{bmatrix}$$

Since p_x is constant:

$$p_x = \text{const}, \quad p_y(t) = -p_x \frac{v}{h} t + \beta \quad \text{for some } \beta \in \mathbb{R}$$

Optimality condition (θ unconstrained):

$$\frac{\partial H}{\partial \theta} = 0 \quad \implies \quad 0 = -p_x v \sin \theta^*(t) + p_y v \cos \theta^*(t)$$

$$\implies \quad \tan \theta^*(t) = \frac{p_y(t)}{p_x} = \frac{-p_x \frac{v}{h} t + \beta}{p_x} = -\frac{v}{h} t + \frac{\beta}{p_x}$$

Setting $\alpha := \frac{\beta}{p_x} \in \mathbb{R}$:

$$\boxed{\tan \theta^*(t) = \alpha - \frac{v}{h} t}$$

Transversality condition (free terminal time):

Substituting $p_y(t_1^*) = -p_x \frac{v}{h} t_1^* + \beta$ (from the costate equation) into H , and using $y(t_1^*) = 0$:

$$\begin{aligned} H^*(t_1^*) &= 1 + p_x \left(v \cos \theta^*(t_1^*) + \frac{v}{h} \underbrace{y(t_1^*)}_{=0} \right) + \left(-p_x \frac{v}{h} t_1^* + \beta \right) v \sin \theta^*(t_1^*) \\ &= 1 + v \left[p_x \cos \theta^*(t_1^*) + \underbrace{\left(-p_x \frac{v}{h} t_1^* + \beta \right)}_{= p_x \tan \theta^*(t_1^*)} \sin \theta^*(t_1^*) \right] \\ &= 1 + v p_x \underbrace{\left(\cos \theta^* + \frac{\sin^2 \theta^*}{\cos \theta^*} \right)}_{= 1 / \cos \theta^*} \\ &= 1 + \frac{v p_x}{\cos \theta^*(t_1^*)} = 0 \\ &\implies p_x = -\frac{\cos \theta^*(t_1^*)}{v} \end{aligned}$$

1.1.2 b)

With $w(y(t)) \equiv \beta$ constant, the Hamiltonian becomes $H = 1 + p_x(v \cos \theta + \beta) + p_y v \sin \theta$, so

$$\dot{p}^*(t) = \begin{bmatrix} -\frac{\partial H}{\partial x} \\ \frac{\partial H}{\partial y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies p_x^* \in \mathbb{R}, p_y^* \in \mathbb{R} \text{ (both constant)}$$

Optimality condition:

$$\frac{\partial H}{\partial \theta} = 0 \implies \tan \theta^*(t) = \frac{p_y^*}{p_x^*} = \text{const} \implies \theta^* = \text{const} := \theta_0$$

Transversality condition (free terminal time):

From the optimality condition $p_y = p_x \tan \theta_0$, substituting into H :

$$\begin{aligned} H^*(t_1^*) &= 1 + p_x(v \cos \theta_0 + \beta) + p_x \tan \theta_0 v \sin \theta_0 \\ &= 1 + p_x \left(v \cos \theta_0 + \beta + \frac{v \sin^2 \theta_0}{\cos \theta_0} \right) \\ &= 1 + p_x \cdot \frac{v + \beta \cos \theta_0}{\cos \theta_0} = 0 \end{aligned}$$

$$\Rightarrow p_x = -\frac{\cos \theta_0}{v + \beta \cos \theta_0}$$

Since θ_0 is constant, the trajectory integrates directly:

$$x(t) = (v \cos \theta_0 + \beta)(t - t_0) + x_0, \quad y(t) = v \sin \theta_0 (t - t_0) + y_0$$

Apply terminal BCs $x(t_1^*) = 0$, $y(t_1^*) = 0$. Let $T := t_1^* - t_0$:

$$(v \cos \theta_0 + \beta)T = -x_0 \quad (1)$$

$$v \sin \theta_0 T = -y_0 \quad (2)$$

Squaring and adding $(1)^2 + (2)^2$: $(vT)^2 = (x_0 + \beta T)^2 + y_0^2$, which expands to

$$(v^2 - \beta^2)T^2 - 2\beta x_0 T - (x_0^2 + y_0^2) = 0$$

Solving by the quadratic formula $T = \frac{2\beta x_0 \pm \sqrt{4[v^2 x_0^2 + (v^2 - \beta^2)y_0^2]}}{2(v^2 - \beta^2)}$:

Case (i) $v > \beta$: leading coefficient > 0 , discriminant $= 4[v^2 x_0^2 + (v^2 - \beta^2)y_0^2] > 0$ always. Taking the positive root:

$$t_1^* - t_0 = \frac{\beta x_0 + \sqrt{v^2 x_0^2 + (v^2 - \beta^2)y_0^2}}{v^2 - \beta^2}$$

No restriction on x_0 .

Case (ii) $v = \beta$: the T^2 term vanishes, leaving a linear equation $-2\beta x_0 T - (x_0^2 + y_0^2) = 0$:

$$t_1^* - t_0 = -\frac{x_0^2 + y_0^2}{2\beta x_0}$$

Requires $x_0 < 0$ for $T > 0$ (ship must start upstream of origin — it cannot make negative headway against the equal current).

Case (iii) $v < \beta$: leading coefficient < 0 ; the discriminant requires $v^2 x_0^2 \geq (\beta^2 - v^2)y_0^2$ for real solutions. Taking the smaller positive root:

$$t_1^* - t_0 = \frac{\beta x_0 + \sqrt{v^2 x_0^2 - (\beta^2 - v^2)y_0^2}}{v^2 - \beta^2}$$

Requires $x_0 < 0$ and $v^2 x_0^2 \geq (\beta^2 - v^2) y_0^2$ (the current is too strong to overcome; ship must start sufficiently upstream).

Please see plot.